

Answering Subset Coverage Counting Queries with Local Differential Privacy

Yuemin Zhang
Hong Kong Polytechnic University
China
ymin.zhang@connect.polyu.hk

Qingqing Ye
Hong Kong Polytechnic University
China
qqing.ye@polyu.edu.hk

Liantong Yu
Hong Kong Polytechnic University
China
23125037r@connect.polyu.hk

Puning Zhao
Sun Yat-sen University
China
zhaopn@mail.sysu.edu.cn

Haibo Hu
Hong Kong Polytechnic University
China
haibo.hu@polyu.edu.hk

ABSTRACT

Answering analytical queries over set-valued data is a fundamental primitive in modern data analysis. In many applications, the data collector is interested in a specific subset of items and seeks to quantify how each user’s itemset overlaps with that subset. We formalize this task as the *subset coverage counting query*: given an *arbitrary* query subset S_Q and a threshold t , the query returns the proportion of users whose itemsets intersect with S_Q in at least t items. To answer these queries in a privacy-preserving manner, a promising way is to enable local differential privacy (LDP), a *de facto* standard for safeguarding individual privacy. However, this way introduces critical challenges: future queries are unknown at the time of data release, and the combinatorial explosion of possible subset queries renders naive enumeration and perturbation infeasible due to excessive LDP noise accumulation.

To address the challenges, we devise a hierarchical subset partitioning tree structure and present a private subset partitioning mechanism that satisfies ϵ -LDP. This mechanism recursively partitions the item domain and uses disjoint user groups to estimate node-level overlap distributions, while leveraging the min-wise hashing technique for accurate intra-node and cross-node approximations. Given an arbitrary query, our mechanism aggregates noisy estimates from only a small number of relevant nodes, avoiding full scans over all noisy reports. Moreover, we develop an adaptive subset decomposition strategy that minimizes an approximated expected error on each node. Comprehensive experiments on different datasets demonstrate the effectiveness of our mechanism.

PVLDB Reference Format:

Yuemin Zhang, Qingqing Ye, Liantong Yu, Puning Zhao, and Haibo Hu. Answering Subset Coverage Counting Queries with Local Differential Privacy. PVLDB, 20(1): XXX-XXX, 2027. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment. Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097. doi:XX.XX/XXX.XX

The source code, data, and/or other artifacts have been made available at <https://github.com/acprojectsac/projsccq>.

1 INTRODUCTION

Nowadays mobile applications produce a considerable amount of set-valued data, such as visited locations and browsing histories. The analysis of such data allows data collectors to enhance their services, bringing benefits to both data collectors and the users. However, such personal data often contains sensitive individual information, such as purchased items, search keywords, and clicked tags. Directly collecting these set-valued data for statistical analysis or model training therefore raises significant privacy concerns.

To address these privacy issues, local differential privacy (LDP) has been recognized as the gold standard for privacy-preserving data analytics and deployed in real-world products by Apple [1], Google [13], and Microsoft [8] for individual privacy protection. Under LDP, each user perturbs their data locally by adding random noise and reports only the sanitized data to the data collector, thereby eliminating the need for a trusted third party.

There is a line of LDP studies on set-valued data. Most existing work focuses on estimating the frequency of a specific item or itemset, or identifying the most frequent items among users, where each user holds a set of private items. In contrast, many real-world applications are less concerned with the exact composition of a user’s set. Instead, the data collector typically cares about a particular subset of items and seeks to quantify how each user’s private set overlaps with that subset. The following are two examples.

- A smart home company wishes to determine how many users own at least three devices from the same brand, in order to decide whether its upcoming product should support interoperability with that brand’s ecosystem.
- A bank needs to know how many accounts triggered at least seven risk control rules in the past quarter, so as to assess the scale of potential fraudulent behavior and guide service adjustments.

In this paper, we formulate such queries as the **subset coverage counting queries**. Specifically, each user possesses a set of items $S_u \subseteq \mathcal{S}$ from the item domain $\mathcal{S}$. Given an *arbitrary* queried subset $S_Q \subseteq \mathcal{S}$ and a threshold $t \in [1, |S_Q|]$, a subset coverage counting query $Q = \langle S_Q, t \rangle$ aims to return the proportion of users who have at least t items within S_Q . To answer this query in the context

of LDP, there are two kinds of adaptations from existing work: (i) a query-specific interactive method that applies standard LDP protocols (e.g., randomized response [26]), where users locally test whether $|S_Q \cap S_u| \geq t$ and report a noisy binary result for each query set S_Q ; and (ii) a set-release approach based on PrivSet [23], where users simply perturb their item sets under a fixed subset size, and the aggregator answers queries by scanning all the noisy sets.

However, both approaches come with clear limitations. The interactive solution must split the privacy budget ϵ across queries to satisfy LDP, thus utility quickly degrades as the query workload grows, since the noise scale is proportional to the query number¹. The set-release solution requires scanning all the perturbed fixed-size subsets for answering each query, resulting in poor utility because of the noise accumulation and the fixed subset size. More importantly, the former implicitly assumes a predefined set of queries, and thus cannot support answering *arbitrary* queries.

Designing an LDP mechanism that addresses the aforementioned limitations while supporting arbitrary queries is non-trivial. The primary challenge stems from the combinatorial explosion of possible subset queries; for instance, when $|S_Q| = 50$ and $t = 5$, the number of distinct queries reaches $\binom{50}{5} = 2,118,760$. Enumerating and privately answering each such query under LDP is infeasible. A natural alternative is to have each user upload a query-agnostic encoding, such that the released information remains sufficiently expressive to answer an arbitrary query, while keeping the accumulation of estimation error within acceptable bounds.

Fortunately, the min-wise hashing (a.k.a., MinHash) technique [2] provides such a decomposable primitive. Each user can compute some MinHash values of her subset S_u using some public permutations, and report the noisy version and a noisy auxiliary statistic $|S_u|$ to the aggregator. When a query arrives, the aggregator computes the MinHash values of the queried subset S_Q using the same permutations, and then aggregates the results between each user's reports and the values of S_Q . The resulting estimation has a closed-form relationship with the overlap between S_u and S_Q .

Note that the above procedure requires scanning all users' noisy reports for each query, and the estimation can be unstable due to the extra noise introduced by auxiliary estimates (e.g., $|S_u|$), especially in large domains and with heterogeneous set sizes. To mitigate such noise accumulation, we devise a hierarchical subset partitioning tree structure where each node corresponds to an item subset and stores some noisy statistics, based on which we then present a private subset partitioning mechanism that integrates MinHash as a query-agnostic approximation component. As such, an arbitrary query can be answered by aggregating estimates from only a handful of relevant tree nodes, rather than scanning all users' noisy reports, which significantly reduces the accumulated noise. We also design a novel adaptive decomposition strategy during tree construction to minimize approximate expected errors.

The key contributions of our study are summarized as follows:

- We formulate the problem of answering subset coverage counting queries under LDP, and present a flexible MinHash-based solution that eliminates the need for per-query user interaction and splitting privacy budget across queries.

- We propose a private subset partitioning mechanism built on a novel hierarchical subset partitioning tree that integrates MinHash, enabling accurate approximation for arbitrary queries while relying mainly on the noisy statistics from a small number of relevant nodes.
- We develop a novel subset decomposition strategy that minimizes the approximated expected error on each node, and adaptively decides the decomposition.
- We conduct comprehensive experiments on different datasets to confirm the superiority of our mechanism in comparison with existing solutions.

Roadmap. Preliminaries and problem statement are presented in Sections 2 and 3. We detail the proposed mechanism with privacy and utility analysis in Sections 4, 5, and 6, and present the experimental results in Section 7. We discuss related works in Section 8, followed by a conclusion in Section 9.

2 PRELIMINARIES

2.1 Local Differential Privacy

LDP [15] enables each user to perturb his data locally, without the need for a trusted data curator. It achieves plausible deniability, which is controlled by a privacy budget ϵ . The following provides a formal definition of LDP.

DEFINITION 1 (ϵ -LDP). *Given a privacy budget $\epsilon > 0$, a randomized mechanism $\mathcal{M} : \mathcal{X} \rightarrow \mathcal{Y}$ provides ϵ -LDP if and only if, for any two inputs $x, x' \in \mathcal{X}$ and any set of possible outputs $\mathcal{Y}$, the following inequality holds: $\Pr[\mathcal{M}(x) \in \mathcal{Y}] \leq e^\epsilon \times \Pr[\mathcal{M}(x') \in \mathcal{Y}]$.*

Similar to centralized settings, LDP enjoys the desired properties of sequential composition and post-processing [12].

THEOREM 1 (SEQUENTIAL COMPOSITION). *Given m randomized algorithms $\mathcal{M}_i$ ($1 \leq i \leq m$), each satisfies ϵ_i -LDP, then the sequence of algorithms $\mathcal{M}_i$ ($1 \leq i \leq m$) collectively satisfies $(\sum \epsilon_i)$ -LDP.*

THEOREM 2 (POST-PROCESSING). *Let $\mathcal{M} : \mathcal{X} \rightarrow \mathcal{Y}$ be a mechanism that satisfies ϵ -LDP, and $\mathcal{F} : \mathcal{Y} \rightarrow \mathcal{Y}^*$ be an arbitrary randomized mapping. Then $\mathcal{F} \circ \mathcal{M} : \mathcal{X} \rightarrow \mathcal{Y}^*$ satisfies ϵ -LDP, where $\circ$ denotes the composition of $\mathcal{F}$ and $\mathcal{M}$.*

2.2 Perturbation Mechanisms

In this subsection, we briefly revisit two classic mechanisms for frequency estimation over categorical data: optimized unary encoding (OUE) [24] and optimized local hashing (OLH) [24].

OUE. The OUE mechanism satisfies ϵ -LDP by encoding an input $x \in \mathcal{X}$ as a $|\mathcal{X}|$ -bit one-hot vector B , where only the x -th bit is set to 1. It then perturbs each bit of B , and outputs B' by: $\Pr[B'[x] = 1] = p^1 = \frac{1}{2}$ if $B[x] = 1$, and $\Pr[B'[x] = 1] = p^0 = \frac{1}{e^\epsilon + 1}$ if $B[x] = 0$. Let count_x denote the number of inputs where $B[x] = 1$, and let $|U|$ be the total number of users. The frequency estimate of x is $f_x^{\text{OUE}} = \frac{\text{count}_x / |U| - p^0}{p^1 - p^0}$, and its variance [24] is $\text{Var}[f_x^{\text{OUE}}] = \frac{4e^\epsilon}{(e^\epsilon - 1)^2 \cdot |U|}$.

OLH. Let $\mathbb{H}$ be a universal hash function family, with each $\text{hash} \in \mathbb{H}$ outputting a value in $[d']$, where $d' = \lceil e^\epsilon + 1 \rceil$ and ϵ is the given privacy budget. The OLH mechanism first encodes the input $x \in \mathcal{X}$ as $h = \text{hash}(x)$, with hash chosen uniformly at random. It then perturbs v , and outputs $y = h$ with probability $p^{\text{olh}} = \frac{e^\epsilon}{d' - 1 + e^\epsilon}$, or returns any value $y \in [d'] \setminus h$ with $q^{\text{olh}} = \frac{1}{d' - 1 + e^\epsilon}$.

¹An alternative is to randomly partition users into groups and each group answers one query. This method also assumes the query number and suffers from the same reason.

For each $x \in X$, let $c_x = |\{u \mid \text{hash}^u(x) = y^u\}|$ denote the number of reports supporting the input as x , where y^u represents the output of user u , perturbed by the randomly chosen hash^u . The frequency estimation of x and its variance [24] are the same as OUE.

2.3 The MinHash Technique

The min-wise hashing (a.k.a., MinHash) [2] was proposed to efficiently approximate the Jaccard similarity $J(S_1, S_2) = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|}$ of two given sets S_1 and S_2 . Jaccard similarity is a commonly used indicator of the similarity between two sets and is widely used to solve the real-world problems such as social networks [6, 28], chemical compounds [29] and information management [5]. The formal definition of MinHash is as follows.

DEFINITION 2 (MINHASH [2]). Let $\mathcal{S} = \{v_1, v_2, \dots\}$ be the universe. Let π be a uniformly random permutation over $\mathcal{S}$. For any set $S \subseteq \mathcal{S}$, the MinHash value ψ of S under π is $\psi_\pi(S) = \min_{v \in S} \pi(v)$.

The probability of two sets to generate the same MinHash value exactly equals the Jaccard similarity of the two sets [19, 27]. Then we have, with L independent permutations $\pi_1, \dots, \pi_L$, the MinHash values are $\psi_1(S_1), \dots, \psi_L(S_1)$, and an estimator of $J(S_1, S_2)$ is $\hat{J}(S_1, S_2) = \frac{1}{L} \sum_{t=1}^L \mathbf{1}[\psi_t(S_1) = \psi_t(S_2)]$, where the indicator function $\mathbf{1}(\cdot)$ equals 1 if the input event happens, or 0 otherwise. It has been proved [19] that $\mathbb{E}(\hat{J}(S_1, S_2)) = J(S_1, S_2)$ and the variance is $J(S_1, S_2) \cdot (1 - J(S_1, S_2))/L$.

3 PROBLEM STATEMENT AND CHALLENGES

3.1 Problem Statement

Let $\mathcal{S} = \{v_1, v_2, \dots\}$ and $\mathcal{U} = \{u_1, u_2, \dots\}$ denote the universal item set and user set. Each user $u \in \mathcal{U}$ holds a subset $S_u \subseteq \mathcal{S}$ locally. A subset coverage counting query is formally defined as follows.

DEFINITION 3 (SUBSET COVERAGE COUNTING QUERY). Given an arbitrary query set $S_Q \subseteq \mathcal{S}$ and a threshold $t \in [1, |S_Q|]$, a query $Q = \langle S_Q, t \rangle$ returns

$$R_Q = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathbb{I}(S_Q, S_u, t),$$

where the indicator function $\mathbb{I}(S_Q, S_u, t) = 1$ if $|S_Q \cap S_u| \geq t$, and $\mathbb{I}(S_Q, S_u, t) = 0$ otherwise.

In this work, we aim to answer arbitrary subset coverage counting queries $Q = \langle S_Q, t \rangle$ with any subset $S_Q \subseteq \mathcal{S}$ and threshold $t \in [1, |S_Q|]$, in the context of LDP. Let $\mathcal{M}$ be a mechanism returning $\hat{R}$, an estimation of the true query result R . The quality of $\mathcal{M}$ is measured by the squared error $(\hat{R} - R)^2$.

3.2 Solutions from Existing Work

To answer a query $Q = \langle S_Q, t \rangle$, there are two types of naive solutions adapted from existing work.

Type I: Query-Specific Interactive Solutions. SeqRR is to apply the randomized response [26] (RR) sequentially. Assume the server knows the number $|Q|$ of the arriving queries Q . The server sends each query $Q = \langle S_Q, t \rangle$ in Q to each user. Each user $u \in \mathcal{U}$ perturbs the result of $\mathbb{I}(S_Q, S_u, t)$ by RR with privacy budget $\epsilon/|Q|$, where S_u denotes the subset owns by u . Then each user uploads the noisy answer to the server for aggregation. By the sequential property of LDP, this method satisfies ϵ -LDP.

An alternative is CRIAD [32], which focuses on answering subset counting queries, namely, the total count of items within a queried subset of the item domain among the user population. In CRIAD, each user encodes her subset into a binary vector with some augmented dummies by filtering the queried subset's items. They perturb the vectors by randomly flipping some bits to 0 (so as to satisfy ϵ -LDP) and upload some sampled indices to the server, which then aggregates the results by summing up the received indices. To support a subset coverage counting query, we can let each user locally check if $|S_Q \cap \hat{S}_u| \geq t$ hold, and then upload the binary result to the server. When there are $|Q|$ queries, one needs to divide the privacy budget by $|Q|$ for each user to achieve ϵ -LDP by sequential composition. Notably, this solution also assumes the number of queries is known and limited.

Type II: Set-Valued Data Collection Solution. PrivSet [23] is designed to estimate item distribution and set cardinality distribution under ϵ -LDP on set-valued data. In PrivSet, each user first truncates or pads their subset to a fixed size, then perturbs the subset by the exponential mechanism [18] with a carefully designed utility function to achieve ϵ -LDP. Finally, each user uploads the perturbed version to the server for estimation. To adapt this method to answer a subset coverage counting query, the server scans each perturbed subset $\hat{S}_u$ to check if $|S_Q \cap \hat{S}_u| \geq t$ for counting.

3.3 Pitfalls of Existing Solutions

CRIAD and SeqRR cannot provide good utility due to two key limitations: 1) the number of queries has to be fixed before estimation, and 2) the privacy budget is divided based on the query number to ensure ϵ -LDP. At first, the number of arriving queries is usually unknown in practice. Second, it may incur excessive LDP noise due to the privacy budget splitting. As for PrivSet, which simply uses the exponential mechanism [18] under a fixed subset size, it is not specifically designed for this query. Directly applying it may lead to poor utility, as for each query, one needs to scan all the noisy answers, leading to large accumulated noise.

Overall, query-specific interactive solutions (CRIAD and SeqRR) suffer from severe noise accumulation due to budget splitting and cannot support arbitrary queries, while the set-valued data collection approach (PrivSet) incurs large accumulated noise. We note that cryptographic approaches such as private set intersection are also ill-suited: they target expensive interactive intersection computation for two given sets; even if adapted to LDP, they also face the privacy budget division issue.

3.4 Straw-Man Solution and Challenges

The main difficulty of answering subset coverage counting queries lies in the the overly large subset combination number. For example, when $|S_Q| = 50$ and $t = 5$, there are $\binom{|S_Q|}{t} = 2,118,760$ candidates, making it infeasible to enumerate and privately estimate them all. Consequently, query-specific interactive solutions (e.g., adapting CRIAD) must spend privacy budget per query and quickly degrade as the workload grows. Set-valued data collection solution (e.g., PrivSet) also may result in poor utility due to the large domain.

Another challenge lies in the dependence of the intersection size $|S_u \cap S_Q|$ on an unseen query subset S_Q , which is unavailable at the time of reporting. A natural and effective solution is to decompose

Algorithm 1: Private MinHash for Subset Coverage Counting Queries on User-Side (PMH-User)

Input: Users U , permutation functions $\{\pi_\ell \mid 1 \leq \ell \leq L\}$, permutation number L , and privacy budget ϵ

Output: The noisy subset size $\widehat{|S_u|}$ and MinHash values $\hat{\psi}^u$

```

1 for  $u \in U$  do
2   User  $u$  perturbs the subset size  $|S_u|$  by the Laplace mechanism with
   privacy budget  $\epsilon/2$ ;
3   for  $1 \leq \ell \leq L$  do
4     User  $u$  uses  $\pi_\ell$  to generate  $\psi_\ell(S_u)$ ;
5     User  $u$  perturbs  $\psi_\ell(S_u)$  by OLH with privacy budget  $\epsilon/2L$ ;
6   User  $u$  uploads  $\widehat{|S_u|}$  and  $\{\hat{\psi}_\ell^u(S_u) \mid 1 \leq \ell \leq L\}$  to the aggregator;
7 return  $\widehat{|S_u|}$  and  $\hat{\psi}^u = \{\hat{\psi}_\ell^u(S_u) \mid 1 \leq \ell \leq L\}$ ;

```

the computation of $|S_u \cap S_Q|$ across the user and the aggregator: each user uploads a small set of encoded and perturbed statistics in a single shot. This shifts the estimation of the intersection size to the query arrival time on the aggregator side. Such a query-agnostic mechanism avoids the aforementioned issues, as it requires no further user interaction and no query-dependent splitting of the privacy budget. Fortunately, MinHash provides a decomposable sketch for this purpose. To compute the intersection size between two subsets S_1 and S_2 , we can compute some MinHash values of two subsets using the same public permutations, then aggregate the value comparisons to obtain an estimate related to the overlap between S_1 and S_2 through $|S_1 \cap S_2|$ and $|S_1 \cup S_2| = |S_1| + |S_2| - |S_1 \cap S_2|$.

This motivates us to design a straw-man solution based on MinHash (see Algorithms 1 and 2), which we call private MinHash (PMH). Assume we have L independent random permutations. Each user first perturbs their subset size $|S_u|$ using the Laplace mechanism [12] using privacy budget $\epsilon/2$. Next, the user runs the MinHash algorithm to obtain L MinHash values, and then applies OLH with privacy budget $\epsilon/2L$ to perturb each value. The user uploads both the perturbed MinHash values and the noisy $\widehat{|S_u|}$ to the aggregator. On the server side, upon receiving an arbitrary query $Q = \langle S_Q, t \rangle$, the aggregator computes the L MinHash values of S_Q using the same permutations. For each user, it then evaluates the probability that

$$\psi_\ell(S_Q) = \hat{\psi}_\ell(S_u)$$

across the L permutations, where $\hat{\psi}_\ell(\cdot)$ denotes the MinHash value from the ℓ -th permutation π_ℓ . This probability, denoted $\hat{J}_u$, serves as an estimation of $\frac{|S_Q \cap S_u|}{|S_Q \cup S_u|}$. Since the MinHash value is perturbed by OLH, we use the following debiased step to obtain an unbiased estimation of J_u . Let $\bar{J}_u = \frac{1}{L} \sum_{\ell=1}^L \mathbf{1}[\psi_\ell(S_Q) = \hat{\psi}_\ell(S_u)]$, we have the unbiased estimation

$$\hat{J}_u = \frac{\bar{J}_u - p_0}{p^{olh} - p_0},$$

where $p_0 = q^{olh} + (p^{olh} - q^{olh})/d'$ and d' is the hash range of OLH. Then the count is incremented if

$$\hat{J}_u \geq \frac{t}{|S_Q| + \widehat{|S_u|} - t},$$

because the threshold condition corresponds to $\frac{|S_Q \cap S_u|}{|S_Q \cup S_u|} = \frac{t}{|S_Q| + |S_u| - t}$.

Finally, the estimated result $\hat{R}_Q$ is obtained by dividing the count by $|U|$. This method satisfies ϵ -LDP, as all values are perturbed before leaving the user, and all aggregator-side operations benefit from the post-processing property.

Algorithm 2: Private MinHash for Subset Coverage Counting Queries on Server-Side (PMH-Server)

Input: User population $|U|$, noisy MinHash values of all users, permutation functions $\{\pi_\ell \mid 1 \leq \ell \leq L\}$, permutation number L , and an arbitrary query $Q = \langle S_Q, t \rangle$

Output: The estimated result $\hat{R}_Q$

```

1  $\hat{R}_Q = 0$ ;
2 for  $u \in U$  do
3    $\bar{J}_u = 0$ ;
4   for  $1 \leq \ell \leq L$  do
5     The aggregator uses  $\pi_\ell$  to generate  $\psi_\ell(S_Q)$ ;
6     if  $\psi_\ell(S_Q) = \hat{\psi}_\ell(S_u)$  then
7        $\bar{J}_u = \bar{J}_u + 1$ ;
8    $\bar{J}_u = (\bar{J}_u/L - p_0)/(p^{olh} - p_0)$ ;
9   if  $\bar{J}_u \geq t/(|S_Q| + \widehat{|S_u|} - t)$  then
10     $\hat{R}_Q = \hat{R}_Q + 1$ ;
11  $\hat{R}_Q = \hat{R}_Q/|U|$ ;
12 return  $\hat{R}_Q$ ;

```

This method enables us to obtain user-level intersection information by decomposing the computation of set intersection size and reusing noisy reports to answer arbitrary queries. However, it suffers from several key limitations:

- The noisy $\widehat{|S_u|}$ can significantly degrade utility.
- Scanning all the reports to answer a query leads to excessive accumulated error.
- The large permutation domain introduces substantial LDP noise.

Therefore, in the following section, we introduce our PSP mechanism to address these limitations. The proposed mechanism offers three key advantages:

- Eliminate the need for noisy $\widehat{|S_u|}$ and enable more accurate estimation on each node.
- Remove the $O(|U| \cdot L)$ dependence by leveraging precomputed node-level results at query time.
- Reduce LDP noise through bounded domain size on each node.

4 PSP: PRIVATE SUBSET PARTITIONING MECHANISM

To answer subset coverage counting queries, in this section, we present a private subset partitioning mechanism (PSP). In PSP, we build a hierarchical *subset partitioning* tree (SP tree), which serves as the building block for answering arbitrary queries. We first introduce our design rationale in Section 4.1.

4.1 Design Rationale

The main challenge in answering subset coverage counting queries stems from the combinatorial explosion of possible subsets. First, predefining and enumerating all the possible queries is infeasible. Second, dividing the privacy budget across queries would incur prohibitively large LDP noise.

Inspired by divide-and-conquer hierarchical indexing, we construct a *subset partitioning* tree (SP tree) by recursively partitioning the item universe into finer-grained subsets to support answering

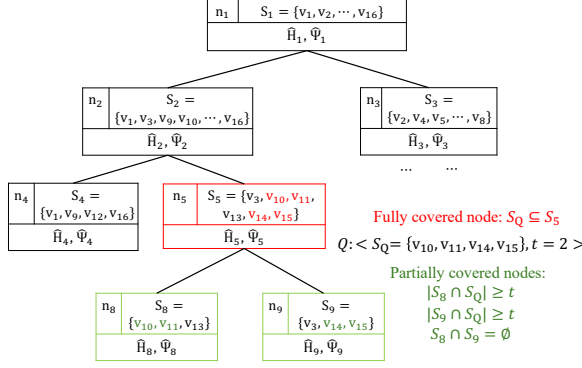


Figure 1: An example of the subset partitioning (SP) tree.

arbitrary queries under LDP. The SP tree is a top-down binary hierarchical structure maintained by the aggregator. Figure 1 shows an example. Let S_u denote the item set held by user u , and let U_k^2 be the users assigned to estimate node n_k . Each node n_k corresponds to an item subset S_k and stores two noisy statistics aggregated from users in U_k^2 : (i) a noisy intersection-size histogram $\hat{H}_k$ estimating the distribution of $|S_u \cap S_k|$ over $u \in U_k^2$, and (ii) perturbed MinHash values $\hat{\Psi}_k$ reported by these users for efficient intra-node approximation at query time. Child nodes partition their parent's subset, but their statistics are not derived from the parent's. For example, in Figure 1, $S_5 = S_8 \cup S_9$, while $(\hat{H}_5, \hat{\Psi}_5)$, $(\hat{H}_8, \hat{\Psi}_8)$, and $(\hat{H}_9, \hat{\Psi}_9)$ are each computed from the reports of users assigned to n_5 , n_8 , and n_9 , respectively. With this design, a query can be answered by aggregating estimates from only a few relevant nodes, rather than all users' noisy reports, thereby substantially reducing noise accumulation.

Meanwhile, dynamic tree construction involves a trade-off when splitting a node. After a split (i.e., node decomposition), a query may span multiple child nodes. Aggregating node-level estimates can introduce additional cross-node approximation error, along with extra error due to user allocation across a larger number of nodes. On the other hand, node decomposition yields finer-grained nodes, thereby providing more detailed information and potentially reducing LDP noise, as the domain size on the new nodes is smaller.

We therefore design a dynamic subset decomposition strategy that, for each node n_k , (i) searches for a candidate *optimal decomposition plan* (i.e., a partition of S_k into two child subsets) that minimizes the expected inter-node approximation error, and (ii) adaptively decides whether to execute the decomposition by comparing the expected decomposition error against the non-decomposition error.

4.2 Workflow of PSP Mechanism

As shown in Figure 2, the PSP mechanism consists of three phases, namely item and item pair distribution estimation, private SP tree construction, and cross-node distribution estimation. Since the private SP tree construction is the core phase, we provide its details in Section 5. Algorithm 3 shows the workflow of PSP.

4.2.1 Estimation Procedure. We now describe the estimation procedure of PSP, which consists of three phases for estimating the necessary statistics and constructing the SP tree for query answering.

Phase I: Item and item-pair distribution estimation. In this phase, we estimate the item-frequency histogram $\hat{H}^{it}$ and the item-pair co-occurrence distribution $\hat{M}^{ip}$, which will later support

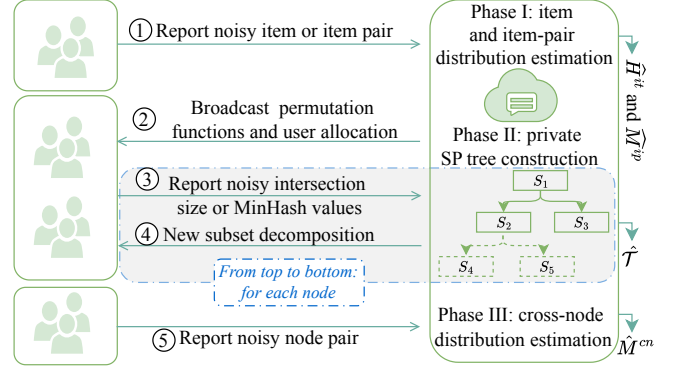


Figure 2: Mechanism overview. Users (left) are divided into three groups. The aggregator (right) first estimates item-(pair) distributions $\hat{H}^{it}$ and $\hat{M}^{ip}$, and then constructs the SP tree $\hat{\mathcal{T}}$ and estimates the cross-node distribution $\hat{M}^{cn}$ using different groups.

intra-node approximation in the query answering procedure and the subset decomposition in Phase II.

The aggregator first partitions users U into three disjoint groups, U^1 , U^2 , and U^3 , for the three phases of the mechanism (line 1 in Algorithm 3), where $|U^1| = |U^3| = \zeta|U|$ and $|U^2| = (1 - 2\zeta)|U|$. Here ζ denotes the user allocation ratio. In this way, each user participates in only one phase and reports only once, avoiding privacy-budget splitting while reducing communication cost.

The users in U^1 are then uniformly split into two subgroups (step ①): one for item-frequency estimation and the other for item-pair estimation. For the first subgroup, each user samples one item from its local set S_u , perturbs it with OUE [24] under privacy budget ϵ (line 5 in Algorithm 3), and sends the perturbed report to the aggregator. For the second subgroup, each user samples one item pair from S_u if $|S_u| \geq 2$, and perturbs it with OUE in the same way (line 7 in Algorithm 3). If $|S_u| = 1$, the user instead samples a dummy pair uniformly from $\binom{S}{2}$ before perturbation.

Finally, the aggregator aggregates the collected OUE reports to obtain the unbiased estimates $\hat{H}^{it}$ and $\hat{M}^{ip}$ (line 8 in Algorithm 3).

Phase II: Private SP tree construction. In this phase, we construct the SP tree $\hat{\mathcal{T}}$, which serves as the core structure for query answering. The aggregator first selects and broadcasts the MinHash permutations, and partitions the users in $U^2 = U \setminus (U^1 \cup U^3)$ into disjoint groups, each assigned to a node in the tree (step ②).

The tree is built in level order. For each node n_k with subset S_k , the assigned users first estimate the noisy intersection-size frequency and MinHash values (step ③), which will later support intra-node approximation. The aggregator then adaptively decides whether and how to split S_k into two child subsets using the optimal new subset decomposition strategy (step ④), so as to reduce the expected decomposition error. As a key component, we provide more detailed techniques of this phase in Section 5.

Phase III: Cross-node distribution estimation. After constructing the tree, we estimate the cross-node co-occurrence distribution $\hat{M}^{cn}$ (step ⑤), since a query may involve items that fall into different nodes and thus requires cross-node co-occurrence information for accurate answering.

For each user $u \in U^3$, the aggregator first maps the user's set S_u to a node-level representation based on tree $\hat{\mathcal{T}}$ (line 10 in Algorithm 3). Let $\mathcal{N}$ denote the set of nodes in $\hat{\mathcal{T}}$. Since an item may belong to multiple nodes, each item $v \in S_u$ is mapped to all nodes in $\hat{\mathcal{T}}$ that contain it. The user then obtains a node-level set $\mathcal{N}_u \subseteq \mathcal{N}$ by taking the union of these mapped nodes over all items in S_u .

Next, the user samples one node pair from $\mathcal{N}_u$: if $|\mathcal{N}_u| \geq 2$, the pair is sampled uniformly from $\binom{\mathcal{N}_u}{2}$; otherwise, a dummy pair is sampled uniformly from $\binom{\mathcal{N}}{2}$ (line 11). The sampled node pair is then perturbed via OUE with privacy budget ϵ and sent to the aggregator (line 12). Finally, the aggregator aggregates the noisy reports to obtain the cross-node co-occurrence matrix $\hat{M}^{cn}$ (line 13).

4.2.2 Query Answering Procedure. After completing the estimation procedure, we describe how to answer a query.

Relevant node selection. Given $Q = \langle S_Q, t \rangle$, we estimate the answer by combining (i) a deepest node n_{fl} that fully covers S_Q and (ii) a set of partially covered nodes $\mathcal{N}_{pt}$. We select n_{fl} in a top-down manner as the deepest node satisfying $S_Q \subseteq S_{n_{fl}}$, and select $\mathcal{N}_{pt}$ bottom-up such that each $n_k \in \mathcal{N}_{pt}$ satisfies $|S_Q \cap S_k| \geq t$ and the selected nodes avoid redundant coverage of the queried items.

EXAMPLE 1. Figure 1 illustrates the node selection for answering $Q = \langle S_Q, t \rangle$, where $S_Q = \{v_{10}, v_{11}, v_{14}, v_{15}\}$ and $t = 2$. We locate the deepest node that fully covers the query set in a top-down manner. In this case, node n_5 satisfies $S_Q \subseteq S_5$ (highlighted in red), while neither of its children fully covers S_Q . Hence, we set $n_{fl} = n_5$. We then select partially covered nodes bottom-up. A node n_j is selected if $|S_j \cap S_Q| \geq t$, and the selected nodes should avoid redundant overlaps on queried items. In Figure 1, $|S_8 \cap S_Q| = 2$ and $|S_9 \cap S_Q| = 2$, corresponding to $\{v_{10}, v_{11}\}$ and $\{v_{14}, v_{15}\}$, respectively. Since these two intersections are disjoint, we obtain $\mathcal{N}_{pt} = \{n_8, n_9\}$ (highlighted in green).

Fully-covered and partially-covered estimators. Based on the selected nodes, we compute a fully-covered estimate $\hat{R}_{full}$ and a partially-covered estimate $\hat{R}_{pt}$. The fully-covered estimate is obtained from the deepest fully covered node n_{fl} , while the partially-covered estimate combines the contributions of the selected partially covered nodes and the cross-node correction terms:

$$\hat{R}_{full} = \hat{R}_{fl}, \quad \hat{R}_{pt} = \sum_{n_k \in \mathcal{N}_{pt}} \hat{R}_k + \sum_{\{n_a, n_b\} \subseteq \mathcal{N}_{pt}} \hat{R}_{a,b}, \quad (1)$$

where $\hat{R}_k$ denotes the query-specific estimate contributed by node n_k , and $\hat{R}_{a,b}$ approximates the joint contribution of a node pair $\{n_a, n_b\}$ to the event $|(S_Q \cap S_a) \cup (S_Q \cap S_b)| \geq t$. In Figure 1, this gives $\hat{R}_{full} = \hat{R}_5$ and $\hat{R}_{pt} = \hat{R}_8 + \hat{R}_9 + \hat{R}_{8,9}$.

Re-weighting node histograms. For a selected node n_k , its intersection-size histogram $\hat{H}_k[i]$ estimates $\Pr(|S_u \cap S_k| = i)$ rather than $\Pr(|S_u \cap S_Q| = i)$. We therefore convert $\hat{H}_k$ into a query-specific estimate by aggregating over all possible intersection sizes:

$$\hat{R}_k = \sum_{i=0}^{|S_k|} \hat{H}_k[i] \cdot \hat{w}_i(n_k, t).$$

Here, $\hat{w}_i(n_k, t)$ estimates the conditional probability that a user satisfies the query threshold given $|S_u \cap S_k| = i$, namely

$$\Pr(|S_u \cap (S_Q \cap S_k)| \geq t \mid |S_u \cap S_k| = i).$$

Algorithm 3: Private Subset Partitioning Tree (PSP)

Input: Users U , user allocation ratio ς , maximum SP tree depth d , and privacy budget ϵ
Output: The noisy item frequency histogram $\hat{H}^{it}$, item-pair co-occurrence distribution $\hat{M}^{ip}$, cross-node co-occurrence distribution $\hat{M}^{cn}$, and SP tree $\hat{\mathcal{T}}$

```

// Phase I: Item(-pair) distribution estimation
1 Partition users  $U$  into  $U^1, U^2, U^3$  with  $|U^1| = |U^3| = \varsigma|U|$  and  $|U^2| = (1 - 2\varsigma)|U|$ ;
2 for  $u \in U^1$  do
3   User  $u$  samples a number  $randint$  from  $\{0, 1\}$  uniformly at random;
4   if  $randint \% 2 = 1$  then
5     User  $u$  perturbs and reports one item sampled uniformly from its subset  $S_u$  via OUE with privacy budget  $\epsilon$ ;
6   else
7     User  $u$  perturbs and reports one item-pair sampled uniformly from  $\binom{S_u}{2}$  if  $|S_u| \geq 2$ , otherwise from  $\binom{\mathcal{N}}{2}$ , via OUE with privacy budget  $\epsilon$ ;
8 The aggregator aggregates the noisy reports to obtain  $\hat{H}^{it}$  and  $\hat{M}^{ip}$ ;
// Phase II: Private SP tree construction
9 Run Algorithm 4 to estimate  $\hat{\mathcal{T}} = SPTreeConstruction(U^2, d, \epsilon)$ ;
// Phase III: Cross-node distribution estimation
10 Each user  $u$  in  $U^3$  aligns their own subsets with all the nodes in  $\hat{\mathcal{T}}$ , denote as  $\mathcal{N}_u \subseteq \mathcal{N}$  and  $\mathcal{N}$  is the node set of  $\hat{\mathcal{T}}$ ;
11 Each user samples a node-pair from  $\mathcal{N}_u$  if  $|\mathcal{N}_u| \geq 2$ , and from  $\mathcal{N}$  otherwise;
12 Each user perturbs the sampled node pair via OUE with privacy budget  $\epsilon$  and uploads to the aggregator;
13 The aggregator aggregates the noisy reports and computes the cross-node co-occurrence distribution  $\hat{M}^{cn}$ ;
14 return  $\hat{H}^{it}, \hat{M}^{ip}, \hat{M}^{cn}$ , and  $\hat{\mathcal{T}}$ ;

```

A naive approximation based on item-wise independence may ignore item correlations. We therefore estimate $\hat{w}_i(n_k, t)$ using the perturbed MinHash reports collected in Phase II.

Estimating $\hat{w}_i(n_k, t)$ via noisy MinHash. Since S_Q and S_k are public, the aggregator computes their MinHash values using the same permutations as the users. Note that each user's subset is aligned to S_k (see Section 5), i.e., $\tilde{S}_u \subseteq S_k \cup \{\perp\}$ and $|\tilde{S}_u \cup S_k| = |S_k| + 1$, where $\perp$ is a public dummy item representing out-of-domain items. Let $\hat{J}(\tilde{S}_u, S_k)$ and $\hat{J}(\tilde{S}_u, S_Q)$ be the noisy Jaccard estimates between $\tilde{S}_u$ and S_k, S_Q , respectively. We recover $|\tilde{S}_u \cap S_k|$ and $|\tilde{S}_u \cap S_Q|$ by solving

$$\hat{J}(\tilde{S}_u, S_k) \approx \frac{|\tilde{S}_u \cap S_k|}{|S_k| + 1}, \quad \hat{J}(\tilde{S}_u, S_Q) \approx \frac{|\tilde{S}_u \cap S_Q|}{|\tilde{S}_u| + |S_Q| - |\tilde{S}_u \cap S_Q|}.$$

Since $\tilde{S}_u \subseteq S_k \cup \{\perp\}$, we have $|\tilde{S}_u| = |\tilde{S}_u \cap S_k| + 1$. We thus first estimate $|\tilde{S}_u \cap S_k|$ from the first equation and then plug it into the second equation to obtain $|\tilde{S}_u \cap S_Q|$. We then estimate $\hat{w}_i(n_k, t)$ by

$$\hat{w}_i(n_k, t) = \frac{\sum_{u \in U_k^{2*}} \mathbf{1}(|\tilde{S}_u \cap S_k| = i, |\tilde{S}_u \cap S_Q| \geq t)}{\sum_{u \in U_k^{2*}} \mathbf{1}(|\tilde{S}_u \cap S_k| = i)},$$

where U_k^{2*} denotes the users assigned to node n_k for MinHash in Phase II, and $\mathbf{1}(\cdot)$ is the indicator function, which equals 1 if the condition in parentheses holds and 0 otherwise. This step consumes no additional privacy budget by the post-processing property.

Cross-node approximation for partially covered nodes. For a node pair $\{n_a, n_b\} \subseteq \mathcal{N}_{pt}$, $\hat{R}_{a,b}$ approximates

$$\Pr(|(S_Q \cap S_a) \cup (S_Q \cap S_b)| \geq t).$$

Algorithm 4: Private SP Tree Construction (*SPTreeConstruction*)

Input: Users U^2 , maximum depth d , and privacy budget ϵ
Output: The noisy SP tree $\hat{\mathcal{T}}$

- 1 The aggregator initializes the binary SP tree $\hat{\mathcal{T}}$ with maximum depth d and randomly assigns users in U^2 to different groups U_k^2 of each node n_k of $\hat{\mathcal{T}}$;
- 2 The aggregator selects the optimal permutation function number L^* via Equation 4 and broadcasts the permutation functions $\{\pi_\ell \mid 1 \leq \ell \leq L^*\}$;
- 3 **for each node** n_k **of** $\mathcal{T}$ **in level order traversal** **do**
- 4 **for user** $u \in U_k^2$ **do**
- 5 User u samples *randint* from $\{0, 1\}$ uniformly at random;
- 6 **if** *randint*%2 = 1 **then**
- 7 User u computes $|S_u \cap S_k|$ between her own subset S_u and this node's subset S_k ;
- 8 User u perturbs $|S_u \cap S_k|$ via OUE with privacy budget ϵ and uploads to the aggregator;
- 9 **else**
- 10 User u aligns her own subset S_u with item domain S_k of n_k to obtain $\tilde{S}_u$;
- 11 Each user u computes L^* MinHash values $\psi^u = [\psi_1^u, \psi_2^u, \dots, \psi_{L^*}^u]$ of $\tilde{S}_u$;
- 12 Each user perturbs each MinHash value via OLH with privacy budget ϵ/L^* to obtain $\hat{\psi}^u$ and uploads it to the aggregator;
- 13 The aggregator computes the intersection size histogram $\hat{H}_k$ via Equation 3 and saves it and all the noisy $\hat{\psi}^u$ in n_k ;
- 14 The aggregator selects the optimal decomposition strategy via Equation 5 and computes the errors of decomposition or stop splitting based on the optimal strategy;
- 15 The aggregator marks n_k as a leaf if it achieves the maximum depth d or the stop splitting error is smaller; otherwise, applies the decomposition;
- 16 The aggregator requires the pruned nodes' (if have) users to estimate the intersection size histograms of the leaf nodes via OUE with privacy budget ϵ and aggregates them;
- 17 **return** $\hat{\mathcal{T}}$;

Since the true distribution is unavailable, we approximate $\hat{R}_{a,b}$ using a Gaussian distribution with mean μ_1 and variance σ_1^2 . Let $S_{Q,a} \cup S_{Q,b} = (S_Q \cap S_a) \cup (S_Q \cap S_b)$, and define $X = \sum_{v \in S_{Q,a} \cup S_{Q,b}} \mathbf{1}\{v \in S_u\}$, which denotes the number of queried items in $S_{Q,a} \cup S_{Q,b}$ covered by a random user u . Then,

$$\mu_1 = \mathbb{E}[X] = \sum_{v \in S_{Q,a} \cup S_{Q,b}} \hat{f}_v,$$

$$\sigma_1^2 = \text{Var}(X) \approx \sum_{v \in S_{Q,a} \cup S_{Q,b}} \hat{f}_v(1 - \hat{f}_v) + 2 \sum_{v_i \in S_{Q,a}, v_j \in S_{Q,b}} (\hat{p}_{v_i v_j}^{cn} - \hat{f}_i \hat{f}_j).$$

Here, $\hat{p}_{v_i v_j}^{cn}$ denotes the estimated co-occurrence probability of a cross-node item pair $\{v_i, v_j\}$, which we approximate by

$$\hat{p}_{v_i v_j}^{cn} = \hat{M}^{cn}[a, b] \cdot \frac{\hat{f}_i}{\sum_{c \in S_{Q,a}} \hat{f}_c} \cdot \frac{\hat{f}_j}{\sum_{d \in S_{Q,b}} \hat{f}_d}.$$

This approximation simulates a worst-case setting in which, conditioned on a user covering items from both nodes, the specific items from $S_{Q,a}$ and $S_{Q,b}$ are drawn independently according to their noisy marginal frequencies.

Inverse-variance aggregation. Let $\hat{R}_Q$ be the final estimate. We combine $\hat{R}_{full}$ and $\hat{R}_{pt}$ via

$$\hat{R}_Q = \frac{\text{Var}_{pt}}{\text{Var}_{full} + \text{Var}_{pt}} \hat{R}_{full} + \frac{\text{Var}_{full}}{\text{Var}_{full} + \text{Var}_{pt}} \hat{R}_{pt}. \quad (2)$$

where $\hat{R}_{pt}$ (resp., $\hat{R}_{full}$) is the estimate obtained from the partially covered (resp., fully covered) node(s), and Var_{pt} (resp., Var_{full}) is its estimation variance.

5 PRIVATE SUBSET PARTITIONING TREE CONSTRUCTION

In this section, we detail how to construct the SP tree. In this tree, each node corresponds an item subset and maintains (i) a noisy intersection-size frequency histogram and (ii) perturbed MinHash reports. The two channels jointly provide the basis for estimating the overlap-size distribution and supporting the coverage approximation when a query set overlaps the node subset. As shown in Algorithm 4, the construction procedure follows a top-down fashion. Specifically, the aggregator initializes a binary SP tree with a root node n_0 associated with the full item domain $S_0 = S$ (line 1). The aggregator then selects the optimal number of permutations L^* (detailed later) and broadcasts the corresponding permutation functions (line 2). Starting from the root, the aggregator processes each node with two steps: (1) noisy frequency and MinHash estimation (lines 7–13), and (2) new subset decomposition (lines 14–16). To allocate users for node-wise estimation, we assume a maximum tree depth d and uniformly partition users in U^2 into 2^d groups, where each group is only assigned to one node in the tree.

5.1 Intersection-Size Frequency and MinHash Value Estimation

Given a node n_k with subset S_k , the aggregator instructs the users assigned to n_k , denoted by U_k^2 , to perturb and upload noisy reports for estimation. The estimation of node n_k consists of two channels. To implement them, we randomly split U_k^2 into two disjoint groups, $U_k^{2'}$ and U_k^{2*} . The first channel, using users in $U_k^{2'}$, produces a noisy intersection-size frequency histogram $\hat{H}_k^o$ that estimates the distribution of overlap sizes $|S_u \cap S_k|$ over users. The second channel, using users in U_k^{2*} , collects perturbed MinHash reports $\hat{\Psi}_k = \{\hat{\psi}^u \mid u \in U_k^{2*}\}$, which can be converted into another histogram $\hat{H}_k^m$ and also supports intra-node approximation. We finally fuse $\hat{H}_k^o$ and $\hat{H}_k^m$ to obtain the final histogram $\hat{H}_k$ stored in node n_k .

5.1.1 Intersection-Size Frequency Estimation. For each user $u \in U_k^{2'}$, the user first computes the intersection size $|S_u \cap S_k|$ locally, where S_u denotes the user u 's itemset, and then perturbs this value using OUE with privacy budget ϵ and uploads the noisy report $|\widehat{S_k \cap S_u}|$ to the aggregator. The aggregator then obtains an unbiased estimate of the OUE-based intersection-size histogram $\hat{H}_k^o$. The idx -th bin of $\hat{H}_k$ corresponds to the estimated frequency of users whose intersection size with S_k equals idx .

5.1.2 MinHash Values Estimation. Although we can obtain unbiased estimates of the overlap size $|S_k \cap S_u|$ via the OUE-based histogram $\hat{H}_k^o$, it can *only* directly answer queries about $|S_k \cap S_u|$. However, subset coverage queries often involve a queried subset S_Q that only partially overlaps the node subset S_k (i.e., $S_Q \subset S_k$), and the quantity needed is $|S_Q \cap S_u|$ rather than $|S_k \cap S_u|$. For example, in Figure 1, consider node n_5 with subset $S_5 = \{v_3, v_{10}, v_{11}, v_{13}, v_{14}, v_{15}\}$, which fully covers the query $Q = \langle S_Q = \{v_{10}, v_{11}, v_{14}, v_{15}\}, t = 2 \rangle$. The histogram $\hat{H}_5^o$ estimates the frequencies of $|S_5 \cap S_u|$, where S_u denotes the itemset of each user $u \in U_5^{2'}$. However, there is a gap between the estimated $|S_5 \cap S_u|$ and the quantity we need, $|S_Q \cap S_u|$, where $S_Q \subset S_5$. A naive workaround is to approximate

$|S_Q \cap S_u|$ from $|S_k \cap S_u|$ by assuming independence among items (e.g., treating each item in S_k as independently appearing in S_u with some marginal probability). This approximation is generally inaccurate for set-valued data, especially when the item occurrences are correlated or when S_Q focuses on a structured subset of S_k .

The PMH approach described in Section 3.4 can avoid this problem. However, **this straightforward approach requires an additional noisy estimate $|S_u|$ from each user under LDP**. The induced noise on $|S_u|$ substantially hurts the utility.

In contrast, in the SP tree, we leverage MinHash values together with the post-processing property of LDP, and avoid adding noise to $|S_u|$ by bounding the union size, so that the utility is improved. Specifically, for each user u , we align the subset S_u with node n_k :

$$\tilde{S}_u = (S_u \cap S_k) \cup \{\perp\},$$

where $\perp$ is a dummy item representing out-of-domain items. Suppose we have L uniformly random permutations $\Pi = \{\pi_1, \pi_2, \dots, \pi_L\}$, which are broadcasted to each user $u \in U_k^{2*}$. Each user u computes the MinHash value $\psi^u = [\psi_1^u, \psi_2^u, \dots, \psi_L^u]$ on its local subset $\tilde{S}_u$, where $\psi_\ell^u = \min_{v \in \tilde{S}_u} \pi_\ell(v)$ is the MinHash value computed under the permutation $\pi_\ell \in \Pi$. The user then perturbs each entry of ψ^u using OLH with privacy budget ϵ/L (so that the overall report satisfies ϵ -LDP), and uploads the perturbed values $\hat{\psi}^u$ to the aggregator. Let $deb(x) = (x - p_0)/(p^{olh} - p_0)$ denote the debiased process function where $p_0 = q^{olh} + (p^{olh} - q^{olh})/d'$ and d' is the OLH hash range. The aggregator then estimates $\hat{J}(\tilde{S}_u, S_k)$ by comparing the user's perturbed MinHash reports with the MinHash values of S_k computed on the aggregator side using the same permutation set Π :

$$\hat{J}(\tilde{S}_u, S_k) = deb\left(\frac{1}{L} \sum_{\ell=1}^L \mathbf{1}[\hat{\psi}_\ell^u = \psi_\ell(S_k)]\right),$$

where the indicator function $\mathbf{1}(\cdot)$ equals 1 if the input event happens, and 0 otherwise. $\hat{\psi}_\ell^u$ denotes the ℓ -th perturbed MinHash value of $\tilde{S}_u$, and $\psi_\ell(S_k)$ is computed on the aggregator side. Since $\tilde{S}_u \subseteq S_k \cup \{\perp\}$, we have $|S_k \cup \tilde{S}_u| = |S_k| + 1$, and thus the (noisy) intersection size can be estimated by

$$|\tilde{S}_u \cap S_k| = |\tilde{S}_u| \cdot \hat{J}(\tilde{S}_u, S_k) = \hat{J}(\tilde{S}_u, S_k) \cdot (|S_k| + 1).$$

Thus, we can obtain another intersection-size frequency histogram $\hat{H}_k^m$ of node n_k via the MinHash estimation channel. Although MinHash is not designed for frequency estimation, it can still provide frequency information between S_k and the aligned user set $\tilde{S}_u$ for each user u . Therefore, we leverage both $\hat{H}_k^o$ and $\hat{H}_k^m$ estimated by OUE and MinHash reports to enhance the utility via

$$\hat{H}_k = \lambda_1 \cdot \hat{H}_k^o + \lambda_2 \cdot \hat{H}_k^m, \quad (3)$$

where $\lambda_1 = \frac{Var_{mh}}{Var_{oue} + Var_{mh}}$ and $\lambda_2 = \frac{Var_{oue}}{Var_{oue} + Var_{mh}}$, and Var_{oue} (resp., Var_{mh}) denotes the variance of OUE (resp., noisy MinHash).

5.1.3 Optimal Permutation Number Selection. The choice of L directly affects the utility of MinHash estimation and exhibits a fundamental tradeoff: a larger L reduces the sampling error of MinHash, while it forces us to split the privacy budget into ϵ/L for perturbation, which increases the OLH noise. Therefore, an optimal L^* should balance these two effects.

The key idea is to select L^* by maximizing the noisy separation between the expected estimation under an arbitrary user's similarity J_u and that at the threshold point. Let τ denote the similarity

corresponding to the intersection threshold t (e.g., $\tau = t/|S_k \cup \tilde{S}_u|$; in our SP tree, $|S_k \cup \tilde{S}_u|$ is bounded as discussed above). Let $Z_u = \frac{1}{L} \sum_{\ell=1}^L X_{u,\ell}$ be the estimated similarity from user u 's perturbed MinHash reports under OLH, where $X_{u,\ell} = \mathbf{1}[\hat{\psi}_\ell^u = \psi_\ell(S_k)]$. We define the score function as the standardized difference:

$$\begin{aligned} \frac{\mathbb{E}[Z_u | J_u] - \mathbb{E}[Z_u | J_u = \tau]}{\sqrt{\text{Var}(Z_u | J_u = \tau)}} &= \frac{\theta(J_u) - \theta_0}{\sqrt{\theta_0(1 - \theta_0)}/L} \\ &= \sqrt{L} \cdot \alpha(\epsilon/L) \cdot \frac{J_u - \tau}{\sqrt{\theta_0(1 - \theta_0)}}, \end{aligned}$$

where $\sum_{\ell} X_{u,\ell} \sim \text{Binomial}(L, \theta(J_u))$, $\theta_0 = \theta(\tau)$, $\theta(J_u) = \Pr[X_{u,\ell} = 1 | J_u] = p_0(\epsilon/L) + \alpha(\epsilon/L) \cdot J_u$, $p_0(\epsilon/L) = q^{olh}(\epsilon/L) + \frac{p^{olh}(\epsilon/L) - q^{olh}(\epsilon/L)}{d'(\epsilon/L)}$, and $\alpha(\epsilon/L) = p^{olh}(\epsilon/L) - p_0(\epsilon/L)$. Here $p^{olh}(\epsilon)$ and $q^{olh}(\epsilon)$ denote the OLH perturbation probabilities under privacy budget ϵ and $d'(\epsilon/L)$ is the OLH hash range under privacy budget ϵ/L . Since for any $\theta_0 \in [0, 1]$, we have $\theta_0(1 - \theta_0) \leq 1/4$. Therefore, for a fixed $|J_u - \tau|$, the magnitude of the score satisfies $\left| \frac{\mathbb{E}[Z_u | J_u] - \mathbb{E}[Z_u | J_u = \tau]}{\sqrt{\text{Var}(Z_u | J_u = \tau)}} \right| \geq 2\sqrt{L} \cdot \alpha(\epsilon/L) \cdot |J_u - \tau|$. Therefore, we can select L^* by maximizing the following factor:

$$L^* = \arg \max_{L \in \mathcal{L}} \sqrt{L} \cdot \alpha(\epsilon/L), \quad (4)$$

where $\mathcal{L} = \{1, 2, \dots, L_c\}$ denotes the candidate set, and L_c is a predefined maximum permutation number.

5.2 New Subset Decomposition

After estimating n_k , we decide whether to split it and, if so, how to partition S_k into two child nodes. Splitting yields smaller domains (hence potentially lower LDP noise and finer statistics), but also allocates fewer users per node and may introduce cross-node approximation error when a query spans multiple children. We therefore (i) search for an *optimal* split of S_k , and (ii) later decide whether executing the split is beneficial.

5.2.1 Optimal Decomposition Strategy Selection. Our objective is to reduce the cross-node approximation error after splitting. This error is most pronounced when queries partially cover multiple children and thus require approximating cross-node co-occurrences under an independence assumption. We therefore seek a partition whose two child subsets are as independent as possible, and quantify such dependence via mutual information [7].

Exhaustively enumerating all partitions ($2^{|S_k|-1} - 1$) is infeasible. We instead restrict candidates to boundary-based splits on an ordered item list Δ_k , which is constructed greedily via a relation score

$$w_{Y,\kappa} = \frac{\hat{M}^{ip}[Y, \kappa]}{\hat{H}^{it}[Y] \cdot \hat{H}^{it}[\kappa]},$$

where $w_{Y,\kappa}$ is the relation score between items v_Y and v_κ and $\hat{H}^{it}[Y]$ and $\hat{M}^{ip}[Y, \kappa]$ are the corresponding noisy item and item-pair co-occurrence frequencies in $\hat{H}^{it}$ and $\hat{M}^{ip}$ from Phase I, so that highly related items are placed close to each other. We initialize Δ_k with

$$v^* = \arg \max_{v_Y \in S_k} \sum_{v_\kappa \in S_k} w_{Y,\kappa},$$

and then iteratively append the item most related to the current last item v_i :

$$v' = \arg \max_{v_Y \in S_k \setminus S_\Delta} w_{Y,i}.$$

After constructing Δ_k , we sweep all candidate boundaries on it. For each split, we measure the dependence between the two child subsets S_{c1} and S_{c2} using the mutual information

$$MI(A; B) = \sum_{\rho \in \{0,1\}} \sum_{\beta \in \{0,1\}} \Pr(A = \rho, B = \beta) \log \frac{\Pr(A = \rho, B = \beta)}{\Pr(A = \rho) \Pr(B = \beta)},$$

$$A = \mathbf{1}(S_u \cap S_{c1} \neq \emptyset), \quad B = \mathbf{1}(S_u \cap S_{c2} \neq \emptyset),$$

where $\mathbf{1}(\cdot)$ is the indicator function. A smaller mutual information indicates a better split.

To compute $MI(A; B)$, it suffices to estimate the joint distribution of (A, B) . Let $X_S \triangleq |S_u \cap S|$. Since S_{c1} and S_{c2} partition S_k , it is enough to estimate the three zero-hit probabilities $p_{00} = \Pr(X_{S_k} = 0)$, $p_{0\cdot} = \Pr(X_{S_{c1}} = 0)$, and $p_{\cdot 0} = \Pr(X_{S_{c2}} = 0)$; the remaining probabilities of (A, B) then follow directly from the marginals. We estimate these zero-hit probabilities using MinHash. The aggregator computes $\hat{J}(\hat{S}_u, S_{c1})$ and $\hat{J}(\hat{S}_u, S_{c2})$ under the same L^* permutations and alignment as above. For $S_c \in \{S_{c1}, S_{c2}\}$, we recover $|\hat{S}_u \cap S_c| = \frac{\hat{J}(\hat{S}_u, S_c)(|\hat{S}_u \cap S_k| + |S_c| + 1)}{1 + \hat{J}(\hat{S}_u, S_c)}$, discretize it into $|S_c| + 1$ bins, and approximate the zero-hit probability by the mass of the 0-th bin.

Finally, for any boundary $\xi \in \{1, \dots, |S_k| - 1\}$, we define $S_{c1} = \text{split}(\Delta_k, \xi) = \{v_1, \dots, v_\xi\}$ and $S_{c2} = S_k \setminus S_{c1}$, compute the corresponding mutual information $MI(A_\xi; B_\xi)$, and choose

$$S_{c1}^* = \text{split}(\Delta_k, \xi^*), \quad S_{c2}^* = S_k \setminus S_{c1}^*, \quad (5)$$

$$\xi^* = \arg \min_{1 \leq \xi \leq |S_k| - 1} MI(A_\xi; B_\xi).$$

5.2.2 Errors of Decomposition and Non-Decomposition. After obtaining the optimal split plan, we decide whether to split n_k or keep it as a leaf. Splitting yields finer-grained (and potentially less noisy) statistics but reduces the number of users per node and introduces cross-node approximation; keeping n_k avoids cross-node effects but may incur larger intra-node approximation error. We choose the option with smaller *approximate expected answering error*.

Given the item subset S_k of node n_k and a query $Q = \langle S_Q, t \rangle$, we evaluate the optimal decomposition strategy $S_k \rightarrow (S_{c1}^*, S_{c2}^*)$ by estimating the expected answering error. We decompose the error into two parts: (i) estimation error due to user sampling and LDP noise, denoted by Err' , and (ii) approximation error, denoted by Err^* . For both cases, the estimation-error part takes the same form:

$$\text{Err}'(\hat{f}_t; m) = \left(1 - \frac{1}{m}\right) \frac{\hat{f}_t(1 - \hat{f}_t) \cdot m}{|U^2|} + \frac{m}{|U^2|} \cdot \frac{\text{Var}_{\text{mh}} \text{Var}_{\text{oue}}}{\text{Var}_{\text{mh}} + \text{Var}_{\text{oue}}},$$

where $\hat{f}_t = \sum_{i=t}^{|S_k|} \hat{w}_i(n_k, t) \hat{H}_k[i]$ is the estimated frequency, as well as an estimate of $\Pr[|S_u \cap S_Q| \geq t]$, the ground-truth tail probability that a random user u satisfies the query predicate. The first term captures sampling variance from partitioning users into m disjoint groups and the second term is the inverse-variance noise of the fused histogram estimator.

With decomposition. If we apply decomposition, the approximate expected answering error Err_1 is

$$\text{Err}_1(Q) = \text{Err}'_1(\hat{f}_t; g + 2) + \text{Err}_1^*(n_k, S_Q, t).$$

where $P_t(S_Q) = \Pr[|S_u \cap S_Q| \geq t]$ denotes the ground-truth tail probability that a random user u satisfies the query predicate. The first term captures the sampling variance induced by allocating users (without replacement) into $g+2$ disjoint groups for node-wise estimation. The second term captures the additional LDP noise in estimating the intersection-size histogram, where the histogram is

obtained by fusing the OUE-based and MinHash-based estimators; hence its variance is the inverse-variance combination.

For the approximation error, let $S_{Q,1} = S_Q \cap S_{c1}$ and $S_{Q,2} = S_Q \cap S_{c2}$. We evaluate the approximation error on each part using the same node-level statistics:

$$\text{Err}_1^*(n_k, S_Q, t) = \frac{1}{2} \sum_{j \in \{1,2\}} \mathbf{1}[|S_{Q,j}| \geq t] \cdot \text{Err}_{in}(n_k, S_{Q,j}, t) + \text{Err}_{cr}(n_k, S_Q, t),$$

where $\text{Err}_{in}(\cdot)$ and $\text{Err}_{cr}(\cdot)$ denote the intra-node and cross-node approximation error, respectively. For part $S_{Q,j}$, we estimate its contribution using the intersection-size histogram $\hat{H}_k$ of node n_k . Specifically, the estimate $\hat{f}_{k,j}$ is a weighted sum over $\hat{H}_k$, where $w_i(n_k, t)$ is the re-weighting factor. Therefore, we approximate the intra-node approximation error by

$$\text{Err}_{in}(n_k, S_{Q,j}, t) = \text{Var}(\hat{f}_{k,j}) = \text{Var}\left(\sum_{i=t}^{|S_k|} \hat{H}_k[i] \cdot \hat{w}_i(n_k, t)\right)$$

$$\approx \frac{1}{|U_k^2|} \left[\sum_{i=t}^{|S_k|} \hat{w}_i(n_k, t)^2 \hat{H}_k[i] - \hat{f}_{k,j}^2 \right],$$

where $|U_k^2|$ is the number of users allocated to node n_k . Recall that we use a Gaussian approximation to estimate the cross-node term $\Pr[|S_{Q,1} \cup S_{Q,2}| \geq t]$ in Equation 1, which depends on the co-occurrence probability $p_{c1,c2}$ between the two child nodes. Let $\eta(\cdot)$ denote the (deterministic) mapping from $p_{c1,c2}$ to the resulting cross-node probability estimate, i.e., $\eta(\hat{p}_{c1,c2})$ under our Gaussian-based post-processing. To quantify the approximation error introduced by uncertainty in $\hat{p}_{c1,c2}$, we use a first-order Taylor approximation:

$$\text{Err}_{cr}(n_k, S_Q, t) = \text{Var}(\eta(\hat{p}_{c1,c2})) \approx (\eta'(\hat{p}_{c1,c2}))^2 \text{Var}(\hat{p}_{c1,c2}).$$

Since $p_{c1,c2}$ is unavailable in Phase II, we use a proxy $\hat{p}_{c1,c2} = (\sum_{v_i \in S_{c1}^*} \hat{f}_i) (\sum_{v_j \in S_{c2}^*} \hat{f}_j)$ from Phase I, and approximate $\text{Var}(\hat{p}_{c1,c2})$ by $\text{Var}(\hat{p}_{c1,c2}) \approx \hat{p}_{c1,c2}(1 - \hat{p}_{c1,c2})/|U_k^2|$ for split decision.

Without decomposition. Similarly, if we do not apply decomposition, the approximate expected answering error is

$$\text{Err}_0(Q) = \text{Err}'_0(\hat{f}_t; g) + \text{Err}_0^*(n_k, S_Q, t),$$

where Err_0^* uses the same variance proxy as the fully-covered case:

$$\text{Err}_0^*(n_k, S_Q, t) \approx \frac{1}{|U_k^2|} \left[\sum_{i=t}^{|S_k|} \hat{w}_i(n_k, t)^2 \hat{H}_k[i] - \hat{f}_t^2 \right].$$

To approximate the expected error, we uniformly sample a query set $\mathcal{Q}_s$ (equal arrival probability) and compute $\text{Err}_1(Q)$ and $\text{Err}_0(Q)$ for all $Q \in \mathcal{Q}_s$. We split n_k iff $\frac{1}{|\mathcal{Q}_s|} \sum_{Q \in \mathcal{Q}_s} \text{Err}_1(Q) < \frac{1}{|\mathcal{Q}_s|} \sum_{Q \in \mathcal{Q}_s} \text{Err}_0(Q)$; otherwise, we stop splitting. If some nodes stop early, their reserved user groups are redistributed to surviving leaves to refine leaf histograms via OUE, improving overall utility.

6 PRIVACY AND UTILITY ANALYSIS

We provide the privacy and utility analysis in this section. Due to the page limit, we defer the detailed proofs to Appendix.

Privacy analysis. We prove that PSP mechanism satisfies ϵ -LDP, as shown in Theorem 3.

THEOREM 3. *The PSP mechanism satisfies ϵ -LDP.*

PROOF SKETCH. In PSP, each user uploads its noisy data perturbed by an LDP mechanism (i.e., OLH or OUE) to the aggregator

only once. All subsequent computations performed by aggregator rely on these sanitized inputs. Thanks to the post-processing property of LDP, the entire PSP mechanism satisfies ϵ -LDP. $\square$

Utility analysis. We analyze the estimation error of the noisy $\hat{R}_Q$ estimated by PSP, which combines the fully-covered answer $\hat{R}_{full}$ and the partially-covered answer $\hat{R}_{pt}$ via Equation 2. Let n_{fl} be the deepest node that fully covers S_Q , and let N_{pt} be the set of selected partially-covered nodes, we have the following theorem.

THEOREM 4. *Given a query $Q = \langle S_Q, t \rangle$. Let $|U_k^2|$ (resp., $|U_{fl}^2|$) denote the numbers of Phase II users assigned to n_k (resp., n_{fl}); $|U^3|$ is the number of Phase III users. For each node n_k , let $\zeta_{min}^k \in (0, 1]$ denote the minimum effective sample fraction of its MinHash bins. The variance of $\hat{R}_Q$ satisfies*

$$Var(\hat{R}_Q) \leq \min \{Var(\hat{R}_{full}), Var(\hat{R}_{pt})\},$$

and $Var(\hat{R}_{full}) \leq \frac{1}{|U_{fl}^2|/2} + \frac{|S_{fl}|+1}{2|U_{fl}^2|\zeta_{min}^{fl}}, Var(\hat{R}_{pt}) \leq \sum_{n_k \in N_{pt}} \left(\frac{1}{|U_k^2|/2} + \frac{|S_k|+1}{2|U_k^2|\zeta_{min}^k} \right) + \sum_{\{n_a, n_b\} \subseteq N_{pt}} \frac{(\eta'(\hat{M}^{cn}[a, b]))^2}{4|U^3|(p-q)^2}$, where p, q are the OUE probabilities, and $\eta(\cdot)$ maps the node-pair co-occurrence probability $\hat{M}^{cn}[a, b]$ to the cross-node correction term (and $\eta'(\cdot)$ is its derivative).

For the ease of analysis, here we assume no pruning and that Phase II users are uniformly partitioned into node groups. With pruning, redistributing unused groups to surviving leaves increases their effective sample sizes, which further tightens the corresponding variance bounds. The query-specific baselines aggregate noisy reports over all users (or repeatedly interact with users per query), which incurs either large domain or privacy budget splitting across queries, both induce excessive LDP noise. In contrast, our solution depends primarily on the number of nodes actually used to answer the query (and the corresponding node-level estimations), yielding much smaller noise accumulation since $|N_{pt}| \ll |U|$.

7 EXPERIMENTS

7.1 Experimental Setting

Datasets. We conduct experiments on three real-world datasets: *UBA*, *POS*, and *MovieLens*. *UBA* is a user behavior dataset from Alibaba [20, 21, 37], one of the largest recommender system datasets collected from real-world industrial scenarios. Each record in *UBA* represents a private user-item interaction. We treat the item brand in each record as the item universe and randomly sample 500,000 records. The item domain size is 5,098. *POS* is a merchant transaction dataset [36], where each transaction is treated as a set of purchased items. The item domain size is 1,657 and the number of users is 435,319. *MovieLens* is a movie rating dataset [14]. We map each movie to its most relevant genome tag and represent each user by the union of tags associated with the movies she rated. This results in 128,493 users and 1,128 items. For all datasets, we filter records whose itemset size is smaller than two.

Baselines. We compare our methods (i.e., PMH and PSP) with three baselines adapted from existing LDP solutions² (i.e., SeqRR, CRIAD, and PrivSet). SeqRR applies randomized response for each

query with privacy budget $\epsilon/|Q|$ (see Section 3.2), where $|Q|$ is the number of queries. The server aggregates the noisy binary responses to estimate the query result. CRIAD [32] is designed for subset counting queries. To answer multiple queries, users are randomly partitioned into $|Q|$ groups, where each group answers one query. For each query, users in the assigned group encode their subsets into binary vectors over the queried subset with augmented dummies and perturb them under ϵ -LDP following the CRIAD protocol. The server aggregates the perturbed vectors, and each user locally checks whether $|S_Q \cap \hat{S}_u| \geq t$ holds to produce the binary response. PrivSet [23] collects perturbed set-valued data under ϵ -LDP. Each user uploads the perturbed set to the aggregator. To answer a query, the aggregator scans each perturbed subset $\hat{S}_u$ and counts the users satisfying $|S_Q \cap \hat{S}_u| \geq t$.

Parameter setting. We set the maximum permutation number $L_c = 16$, maximum depth of the SP tree $d = 8$, and user allocation ratio $\varsigma = 0.2$ in experiments. We use the default hyperparameter settings from the original papers for all baselines.

Metric. We evaluate the utility of the mechanisms using the mean square error (MSE) metric. Given a query set Q , the MSE is computed as $MSE(Q) = \sum_{Q \in Q} \frac{(\hat{R}_Q - R_Q)^2}{|Q|}$, where R_Q and $\hat{R}_Q$ are the ground truth answer and the noisy estimate for the query Q . We randomly generate 2,000 queries, each of which $Q = \langle S_Q, t \rangle$ has a random itemset S_Q and a uniformly sampled threshold $1 < t < |S_Q|$. We calculate the MSE of these queries, where smaller scores indicate better performance. All the mechanisms are executed 50 times and the average is plotted.

7.2 Evaluation Results

Overall results on utility. We first evaluate the performance of our proposed PSP mechanism against the baselines on three datasets. We vary the privacy budget ϵ from $\{0.1, 0.5, 1, 2, 3, 4, 5, 6\}$ and the queried set size $|S_Q| \in \{20, 50, 100\}$. The results are shown in Figure 3, which demonstrate the superiority of PSP. PrivSet always has the highest MSE because the fixed set size substantially restricts the answering accuracy. SeqRR and CRIAD also perform poorly since their noise scale grows with the number of queries. Although PMH performs better in some cases, the noisy estimation of each user's set size significantly degrades its accuracy. In contrast, PSP consistently outperforms PMH and all baselines, demonstrating the effectiveness of our MinHash-based approximation and optimal subset decomposition strategy.

Impact of varying threshold proportions. We study the impact of different threshold proportions $t/|S_Q|$ under $\epsilon \in \{0.5, 1, 2, 4\}$. Figure 4 reports the results when $t/|S_Q| \in \{10\%, 25\%, 50\%\}$ under $|S_Q| = 50$. As ϵ increases, the MSE of all methods decreases since less perturbation noise is introduced. Across all datasets and parameter settings, PSP consistently achieves the lowest error.

When the threshold proportion increases, the overall estimation becomes more difficult for most baselines. This is because a larger threshold requires more accurate estimation of the overlap between the user set and the queried subset. Methods such as SeqRR and CRIAD suffer from large variance due to query-level perturbation, while PrivSet performs poorly because the fixed subset size limits the representation accuracy of the original sets. In

²SeqRR and CRIAD require a predefined query set Q , which we assume to be known to SeqRR and CRIAD to make them comparable.

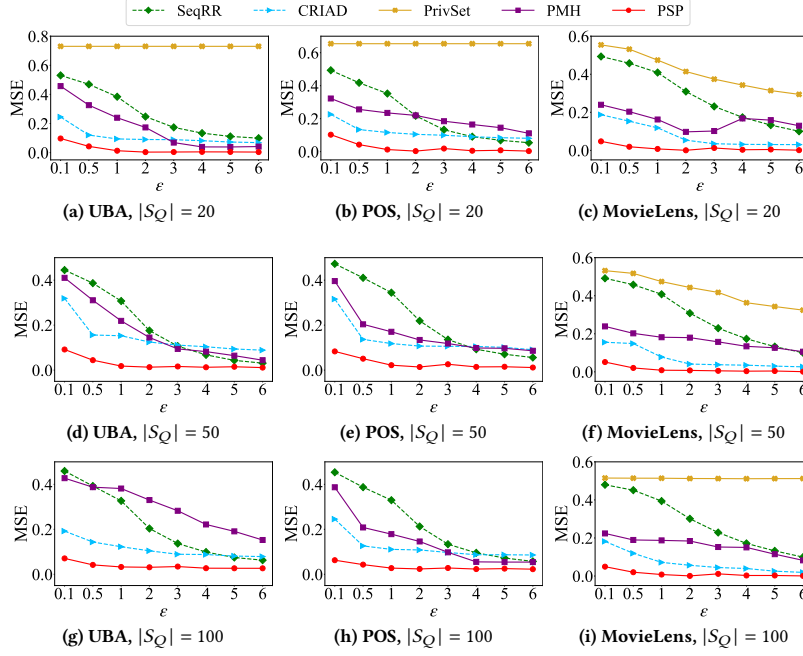


Figure 3: Overall utility on varying privacy budgets ϵ .

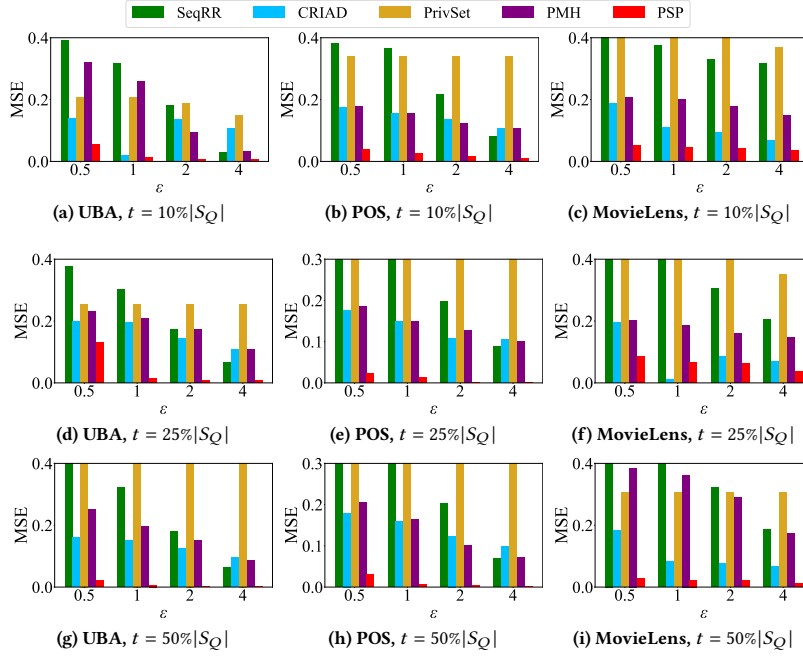


Figure 4: Impact study on varying threshold proportions under varying ϵ (with $|S_Q| = 50$).

contrast, PSP maintains stable performance under different thresholds, demonstrating the robustness of the proposed MinHash-based approximation and subset decomposition strategy.

Impact of different maximum depths. We then study the impact of the maximum depth tree depth of the SP tree on the performance of PSP. Figure 5 reports the results when $d \in \{4, 6, 8, 10\}$. Overall, increasing the depth slightly improves the accuracy when ϵ is small, since a deeper tree provides finer-grained partitioning of the item space. However, with increasing d , the performance

improvement becomes marginal. This is because deeper trees introduce additional noise to node statistics. Therefore, a moderate tree depth achieves the best trade-off between partition granularity and noise accumulation.

Impact of optimal decomposition. We further evaluate the effectiveness of the optimal decomposition strategy used in PSP. Figure 6 compares the PSP with a variant without optimal partition, i.e., applying uniformly random partition. The results show that the proposed decomposition strategy consistently reduces the

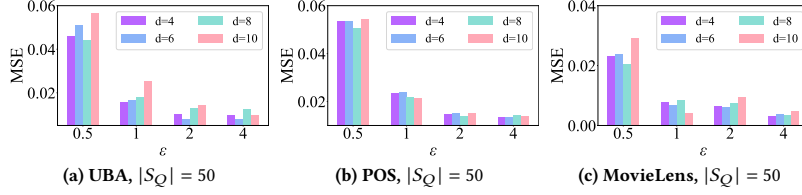


Figure 5: Impact study on different maximum depths of the SP tree under varying ϵ .

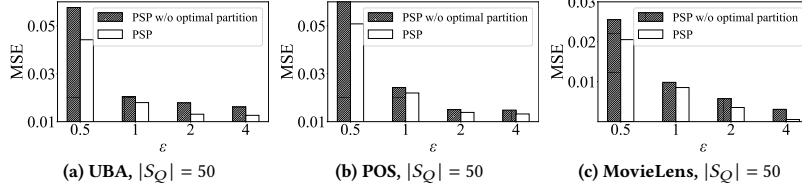


Figure 6: Impact study on the optimal decomposition strategy of the SP tree under varying ϵ .

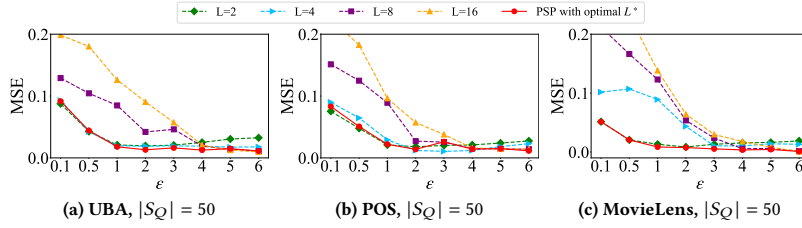


Figure 7: Impact study on the optimal permutation number under varying ϵ .

estimation error across all datasets. The improvement is more significant when the privacy budget is small, because selecting the optimal partition effectively reduces unnecessary decomposition noise. This demonstrates that the proposed optimization is critical for improving the utility of the SP tree.

Impact of the optimal permutation number. We evaluate the impact of the number of permutations L used for MinHash. Figure 7 compares different fixed choices of $L \in \{2, 4, 8, 16\}$ with our adaptive selection strategy L^* . The results show that the optimal value of L varies with the privacy budget ϵ . When ϵ is small, using a large number of permutations introduces more perturbation noise and degrades the accuracy. As ϵ increases, larger L provides a more accurate MinHash approximation and improves the estimation accuracy. Our adaptive strategy automatically selects the near-optimal L^* for PSP across different privacy budgets and datasets, consistently achieving the lowest or near-lowest MSE.

8 RELATED WORK

Differential privacy (DP) [10] has become a de facto standard for privacy protection and has been widely used in analyzing streaming data [9, 11], data publishing [3, 4, 34], and learning tasks [30, 33]. However, DP typically assumes a trusted server. Local differential privacy (LDP) [15, 16] removes this assumption by letting users perturb data locally before reporting, and has been adopted in practice for various federated data analytics scenarios [1, 8, 13, 31].

Existing LDP studies on set-valued data mainly focus on frequent itemset mining [25] and heavy hitter identification [17, 22, 35]. Wang *et al.* [23] proposed PrivSet to estimate item frequencies and set-cardinality distributions under ϵ -LDP. Yet, adapting PrivSet to subset coverage counting queries requires the server to scan (noisy)

reported subsets, leading to high query-time cost and extremely large noise accumulation. The most relevant work is CRIAD [32], which is designed for answering a *single* counting query: the total number of occurrences of items within a queried subset across the population. In CRIAD, each user perturbs and uploads a query-dependent binary vector (with some dummy items) to the aggregator. To answer a subset coverage counting query $\langle S_Q, t \rangle$, one can let each user test whether the sum of 1 reports on the perturbed vector's queried positions is larger than t . However, when there are many queries, the privacy budget must be split across queries due to sequential composition, which quickly degrades utility. An alternative is to assign the users into query-number groups and require each group's users to answer one query, also leading to poor utility when the number is large (e.g., 1000). More importantly, CRIAD implicitly assumes the query workload is known and limited in advance, and thus cannot support answering *arbitrary* queries.

9 CONCLUSION

In this paper, we formulated and studied the problem of answering subset coverage counting queries under local differential privacy. Existing solutions either require per-query interaction with privacy budget splitting or suffer from large noise accumulation. To address this challenge, we proposed PSP, a private subset partitioning mechanism built on a novel hierarchical subset partitioning tree that leverages MinHash-based approximation to support arbitrary queries efficiently while relying mainly on only a few noisy estimates of relevant nodes. In addition, we introduced an adaptive subset decomposition strategy that dynamically determines the optimal partitioning structure. Extensive experiments on different datasets confirm the effectiveness of our proposed mechanism.

REFERENCES

- [1] Differential Privacy Team Apple. 2017. Learning with Privacy at Scale. In *Apple Machine Learning Journal*.
- [2] Andrei Z Broder, Moses Charikar, Alan M Frieze, and Michael Mitzenmacher. 1998. Min-Wise Independent Permutations. In *STOC*. 327–336.
- [3] Rui Chen, Gergely Acs, and Claude Castelluccia. 2012. Differentially Private Sequential Data Publication via Variable-Length N-Grams. In *CCS*. 638–649.
- [4] Rui Chen, Qian Xiao, Yu Zhang, and Jianliang Xu. 2015. Differentially Private High-Dimensional Data Publication via Sampling-Based Inference. In *Proceedings of the 21st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (SIGKDD)*. 129–138.
- [5] Ludmila Cherkasova, Kave Eshghi, Charles B. Morrey, Joseph Tucek, and Alstair Veitch. 2009. Applying Syntactic Similarity Algorithms for Enterprise Information Management. In *KDD*. 1087–1096.
- [6] Flavio Chierichetti, Ravi Kumar, Silvio Lattanzi, Michael Mitzenmacher, Alessandro Panconesi, and Prabhakar Raghavan. 2009. On Compressing Social Networks. In *KDD*. 219–228.
- [7] Thomas M Cover. 1999. *Elements of Information Theory*. John Wiley & Sons.
- [8] Bolin Ding, Janardhan Kulkarni, and Sergey Yekhanin. 2017. Collecting Telemetry Data Privately. In *NeurIPS*. 3574–3583.
- [9] Wei Dong, Qiyao Luo, and Ke Yi. 2023. Continual Observation under User-Level Differential Privacy. In *IEEE SP*. 2190–2207.
- [10] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *Proceedings of the 3rd Conference on Theory of Cryptography (TCC)*. 265–284.
- [11] Cynthia Dwork, Moni Naor, Toniann Pitassi, and Guy N Rothblum. 2010. Differential Privacy under Continual Observation. In *STOC*. 715–724.
- [12] Cynthia Dwork, Aaron Roth, et al. 2014. The Algorithmic Foundations of Differential Privacy. In *TCS*, Vol. 9. 211–407.
- [13] Ulfar Erlingsson, Vasyl Pihur, and Aleksandra Korolova. 2014. Rappor: Randomized Aggregatable Privacy-Preserving Ordinal Response. In *Proceedings of the 21st ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1054–1067.
- [14] F Maxwell Harper and Joseph A Konstan. 2015. The MovieLens Datasets: History and Context. In *ACM Transactions on Interactive Intelligent Systems (TIIS)*, Vol. 5. 1–19.
- [15] Shiva Prasad Kasiviswanathan, Homin K Lee, Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. 2011. What Can We Learn Privately?. In *SICOMP*, Vol. 40. 793–826.
- [16] Ninghui Li and Qingqing Ye. 2019. Mobile Data Collection and Analysis with Local Differential Privacy. In *Proceedings of the IEEE 20th International Conference on Mobile Data Management (MDM)*. 4–7.
- [17] Xiaochen Li, Weiran Liu, Jian Lou, Yuan Hong, Lei Zhang, Zhan Qin, and Kui Ren. 2024. Local Differentially Private Heavy Hitter Detection in Data Streams with Bounded Memory. In *Proceedings of the 2024 ACM Conference on Management of Data (SIGMOD)*. 1–27.
- [18] Frank McSherry and Kunal Talwar. 2007. Mechanism Design via Differential Privacy. In *FOCS*. IEEE, 94–103.
- [19] Michael Mitzenmacher, Rasmus Pagh, and Ninh Pham. 2014. Efficient Estimation for High Similarities using Odd Sketches. In *WWW*. 109–118.
- [20] Qi Pi, Weijie Bian, Guorui Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Practice on Long Sequential User Behavior Modeling for Click-Through Rate Prediction. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*. 2671–2679.
- [21] Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based User Interest Modeling with Lifelong Sequential Behavior Data for Click-Through Rate Prediction. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management (CIKM)*. 2685–2692.
- [22] Zhan Qin, Yin Yang, Ting Yu, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2016. Heavy Hitter Estimation over Set-Valued Data with Local Differential Privacy. In *Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 192–203.
- [23] Shaowei Wang, Liusheng Huang, Yiwen Nie, Pengzhan Wang, Hongli Xu, and Wei Yang. 2018. Privset: Set-Valued Data Analyses with Locale Differential Privacy. In *IEEE INFOCOM*. 1088–1096.
- [24] Tianhao Wang, Jeremiah Blocki, Ninghui Li, and Somesh Jha. 2017. Locally Differentially Private Protocols for Frequency Estimation. In *USENIX Security*. 729–745.
- [25] Tianhao Wang, Ninghui Li, and Somesh Jha. 2018. Locally Differentially Private Frequent Itemset Mining. In *2018 IEEE Symposium on Security and Privacy (SP)*. 127–143.
- [26] Stanley L Warner. 1965. Randomized Response: A Survey Technique for Eliminating Evasive Answer Bias. In *Journal of the American Statistical Association (JASA)*, Vol. 60. 63–69.
- [27] Wei Wu, Bin Li, Ling Chen, Junbin Gao, and Chengqi Zhang. 2022. A Review for Weighted MinHash Algorithms. In *TKDE*, Vol. 34. 2553–2573.
- [28] Wei Wu, Bin Li, Ling Chen, and Chengqi Zhang. 2018. Efficient Attributed Network Embedding via Recursive Randomized Hashing. In *IJCAI*. 2861–2867.
- [29] Wei Wu, Bin Li, Ling Chen, Xingquan Zhu, and Chengqi Zhang. 2018. K -Ary Tree Hashing for Fast Graph Classification. In *TKDE*, Vol. 30. 936–949.
- [30] Zihang Xiang, Tianhao Wang, and Di Wang. 2024. Preserving Node-Level Privacy in Graph Neural Networks. In *IEEE SP*. 4714–4732.
- [31] Qingqing Ye, Haibo Hu, Xiaofeng Meng, and Huadi Zheng. 2019. PrivKV: Key-Value Data Collection with Local Differential Privacy. In *Proceedings of the 40th IEEE Symposium on Security and Privacy (SP)*. 317–331.
- [32] Qingqing Ye, Liantong Yu, Kai Huang, Xiaokui Xiao, Weiran Liu, and Haibo Hu. 2025. From Randomized Response to Randomized Index: Answering Subset Counting Queries with Local Differential Privacy. In *IEEE SP*. 3877–3891.
- [33] Shuzhan Ye, Lu Chen, Zhikun Zhang, Yunjun Gao, Yuxiang Wang, and Xiaoliang Xu. 2025. PrivAGS: Differentially Private Attributed Graph Synthesis. In *SIGMOD*, Vol. 3. 1–25.
- [34] Jun Zhang, Graham Cormode, Cecilia M Procopiuc, Divesh Srivastava, and Xiaokui Xiao. 2017. PrivBayes: Private Data Release via Bayesian Networks. In *ACM Transactions on Database Systems (TODS)*, Vol. 42. 1–41.
- [35] Yuemin Zhang, Qingqing Ye, and Haibo Hu. 2025. Federated Heavy Hitter Analytics with Local Differential Privacy. In *Proceedings of the 2025 ACM Conference on Management of Data (SIGMOD)*, Vol. 3. Article 42, 27 pages.
- [36] Zijian Zheng, Ron Kohavi, and Llew Mason. 2001. Real World Performance of Association Rule Algorithms. In *KDD*. 401–406.
- [37] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep Interest Evolution Network for Click-Through Rate Prediction. In *Proceedings of the 33rd AAAI Conference on Artificial Intelligence (AAAI)*. 5941–5948.

A APPENDIX

A.1 Privacy Analysis

THEOREM 5. *The PSP mechanism satisfies ϵ -LDP.*

PROOF. In the PSP mechanism, users are first divided into three groups U^1 , U^2 , and U^3 , which does not raise privacy concerns. Each user in a group receives a candidate domain for perturbation and, given the privacy budget ϵ , perturbs their value via OUE or OLH. We first prove that the perturbation satisfies ϵ -LDP, and then show that candidate domain construction does not consume any additional privacy budget.

In Phase I, each user in U^1 perturbs one of their items or an item pair via OUE with privacy budget ϵ . For the users who upload the noisy item, they first randomly sample one item of their subsets, then perturb and upload to the aggregator. Since the OUE satisfies ϵ -LDP, the resulting frequency distribution benefits from the post-processing theorem. For the users who estimate the item-pair distribution, they also follows the similar way to sample an item-pair, thus also satisfy ϵ -LDP and enjoy the post-processing theorem.

In Phase II, users in U^2 are further divided into groups for estimating noisy MinHash values or histograms. Each user perturbs L MinHash values by ϵ/L via OLH or the intersection size by OUE with ϵ . For the users who upload L noisy MinHash values, the sequential composition theorem guarantees that they satisfy ϵ -LDP. For the users who estimate the intersection sizes, they also satisfy ϵ -LDP. Further operations on the aggregator side are based on the noisy data, thus, these operations do not consume additional privacy budget via the post-processing property.

In Phase III, users are used to estimate the node-pair distribution or intersection size by OUE with ϵ . Similar to that in the previous phases, the perturbations in this phase also satisfies ϵ -LDP. Thus, we complete the proof. $\square$

A.2 Utility Analysis

LEMMA 1. Fix a node n_k and a query $Q = \langle S_Q, t \rangle$. Let U_k^2 be the Phase II user set assigned to n_k and it is uniformly split into two disjoint groups: $U_k^{2'}$ for estimating the OUE-based histogram $\hat{H}_k^o$ and U_k^{2*} for estimating the MinHash-based factors, with $|U_k^{2'}| = |U_k^{2*}| = |U_k^2|/2$. Then

$$\text{Var}(\hat{R}_k(Q)) \leq \frac{1}{|U_k^{2'}|} + \sum_{i=0}^{|S_k|} \mathbb{E}[\hat{H}_k^o[i]^2] \cdot \text{Var}(\hat{w}_i(n_k, t)). \quad (6)$$

Moreover, let $N_{k,i}$ denote the effective number of users in U_k^{2*} used to estimate $\hat{w}_i(n_k, t)$. Suppose that $N_{k,i} \geq |U_k^{2*}| \zeta_i^k$ and $0 < \zeta_i^k \leq 1$. Then

$$\begin{aligned} \text{Var}(\hat{w}_i(n_k, t)) &\leq \frac{1}{4|U_k^{2*}| \zeta_i^k}, \\ \text{Var}(\hat{R}_k(Q)) &\leq \frac{1}{|U_k^{2'}|} + \sum_{i=0}^{|S_k|} \frac{\mathbb{E}[\hat{H}_k^o[i]^2]}{4|U_k^{2*}| \zeta_i^k}. \end{aligned} \quad (7)$$

Let $\zeta_{\min}^k = \min_i \zeta_i^k$ and $0 < \zeta_{\min}^k \leq 1$. Using the coarse bound $\mathbb{E}[\hat{H}_k^o[i]^2] \leq 1$, we obtain

$$\text{Var}(\hat{R}_k(Q)) \leq \frac{1}{|U_k^{2'}|} + \frac{|S_k| + 1}{4|U_k^{2*}| \zeta_{\min}^k}.$$

With $|U_k^{2'}| = |U_k^{2*}| = |U_k^2|/2$, this becomes

$$\text{Var}(\hat{R}_k(Q)) \leq \frac{1}{|U_k^2|/2} + \frac{|S_k| + 1}{2|U_k^2| \zeta_{\min}^k}. \quad (8)$$

PROOF. By the law of total variance, $\text{Var}(\hat{R}_k) = \mathbb{E}[\text{Var}(\hat{R}_k | \hat{w})] + \text{Var}(\mathbb{E}[\hat{R}_k | \hat{w}])$. Conditioned on $\hat{w}$, $\hat{R}_k$ is a weighted sum of the histogram bins. Using a coarse bound for the OUE histogram estimator yields $\mathbb{E}[\text{Var}(\hat{R}_k | \hat{w})] \leq 1/|U_k^{2'}|$. For the second term, applying $\text{Var}(\sum_i a_i X_i) \leq \sum_i a_i^2 \text{Var}(X_i)$ gives Equation 6. Conditioned on the effective bin size $N_{k,i}$, $\hat{w}_i$ is a (smoothed) binomial proportion, and hence $\text{Var}(\hat{w}_i(n_k, t)) \leq \frac{1}{4N_{k,i}} \leq \frac{1}{4|U_k^{2*}| \zeta_i^k}$. Substituting this bound into Equation 6 gives Equation 7. Letting $\zeta_{\min}^k = \min_i \zeta_i^k$ and using $|U_k^{2'}| = |U_k^{2*}| = |U_k^2|/2$ gives Equation 8. $\square$

LEMMA 2. Let U^3 be the Phase III user set and let $|U^3|$ denote its size. For a node pair $\{n_a, n_b\}$, let $p_{a,b}$ be the true node-pair co-occurrence probability and let $\hat{M}^{cn}[a, b]$ be its OUE estimate obtained from $|U^3|$ users. Let $p = \frac{1}{2}$ and $q = \frac{1}{e+1}$ be the OUE probabilities. Define $u = q + (p - q)\hat{M}^{cn}[a, b]$. Then

$$\text{Var}(\hat{p}_{a,b}) = \frac{u(1-u)}{|U^3|(p-q)^2} \leq \frac{1}{4|U^3|(p-q)^2}. \quad (9)$$

Moreover, if the cross-node correction is $\hat{R}_{a,b}(Q) = \eta(\hat{M}^{cn}[a, b])$ for a differentiable mapping $\eta(\cdot)$, then by the first-order Delta method,

$$\text{Var}(\hat{R}_{a,b}(Q)) \approx (\eta'(\hat{M}^{cn}[a, b]))^2 \text{Var}(\hat{M}^{cn}[a, b]) \leq \frac{(\eta'(\hat{M}^{cn}[a, b]))^2}{4|U^3|(p-q)^2}. \quad (10)$$

PROOF. Equation 9 is the standard variance of the unbiased OUE frequency estimator for a single category, and $u(1-u) \leq 1/4$ gives the upper bound. Equation 10 follows from the Delta method: $\eta(\hat{p}) \approx \eta(p) + \eta'(p)(\hat{p} - p)$. $\square$

THEOREM 6. Let n_{fl} be the deepest node that fully covers S_Q and let N_{pt} be the set of selected partially-covered nodes. Assume Phase II user groups are disjoint across nodes. Then the variances of $\hat{R}_{full}(Q)$ and $\hat{R}_{pt}(Q)$ satisfy

$$\text{Var}(\hat{R}_{full}(Q)) \leq \frac{1}{|U_{fl}^2|/2} + \frac{|S_{fl}| + 1}{2|U_{fl}^2| \zeta_{\min}^{fl}}, \quad (11)$$

$$\text{Var}(\hat{R}_{pt}(Q)) \leq \sum_{n_k \in N_{pt}} \left(\frac{1}{|U_k^2|/2} + \frac{|S_k| + 1}{2|U_k^2| \zeta_{\min}^k} \right) + \sum_{\{n_a, n_b\} \subseteq N_{pt}} \frac{(\eta'(\hat{M}^{cn}[a, b]))^2}{4|U^3|(p-q)^2}. \quad (12)$$

Moreover, for the inverse-variance combination in Equation 2,

$$\text{Var}(\hat{R}_Q) \leq \min\{\text{Var}(\hat{R}_{full}), \text{Var}(\hat{R}_{pt})\}.$$

PROOF. Apply Lemma 1 to n_{fl} and each $n_k \in N_{pt}$ using the corresponding $\zeta_{\min}^k$ bounds to obtain Equations 11 and 12. For $\hat{R}_{pt}$, variances add up across disjoint Phase II user groups, and each cross-node correction term follows Lemma 2. Finally, the inequality $\text{Var}(\hat{R}_Q) \leq \min\{\text{Var}(\hat{R}_{full}), \text{Var}(\hat{R}_{pt})\}$ follows from inverse-variance weighting of two approximately independent estimators. $\square$